FPT University

Class IC2009

Moving Average Noise Filter Using MATLAB

DIGITAL SIGNAL PROCESSING PROJECT REPORT

Course: DGT302 - Digital Signal Processing

Group members:

Nguyễn Ngô Anh Vũ - SE203026

Dương Đức Thịnh - SE203340

Trần Nguyễn Hải Đăng - SE202009

Instructor: Phạm Thế Vinh

Report prepared on 1 November 2026

Abstract

This project examines a causal moving average finite impulse response filter for reducing additive Gaussian noise in a sampled sinusoidal signal. The supplied MATLAB script generates a 5 Hz sine wave at a sampling frequency of 1000 Hz over an endpoint-inclusive record from 0 to 2 s. It adds Gaussian noise with a population standard deviation of 0.6 and applies a 31-sample moving average using filter(b,1,x_noisy). The source script defines the implemented baseline; the README instead describes a 15-sample window.

The numerical results and figures were obtained from a Python reproduction of the signal model and causal filter specified by the MATLAB script. NumPy RandomState with seed 1 generated the noise realization. The Gaussian samples are specific to this reproduction and are not claimed to match those generated by MATLAB rng(1) and randn.

On the common target interval n = 30 through 1985, the measured input noise power is 0.365296 and the separated filtered noise power is 0.010309, in squared amplitude units. The input and output signal-to-noise ratios are 1.4247 dB and 16.5735 dB, respectively. Reconstruction mean squared error decreases from 0.365296 before filtering to 0.116688 without alignment and 0.009557 after explicit delay compensation. The latter is an offline comparison, not a change to the implemented filter.

The filter suppresses random fluctuations but introduces a theoretical delay of 15 samples, or 15 ms, and a 5 Hz amplitude gain of 0.960986. The analysis separates filtered noise from deterministic attenuation and delay, explains startup under zero initial conditions, and identifies frequency selectivity and finite-record uncertainty as limitations.

Table of Contents

Abstract 2
1 Introduction 4
2 Project Objectives 4
3 Theoretical Background 5
3.1 Sampling and the original signal 5
3.2 Additive Gaussian noise 5
3.3 Causal moving average filtering 5
3.4 Impulse and frequency responses 6
3.5 Smoothing and delay 6
3.6 Startup under zero initial conditions 6
4 System Design and Simulation Parameters 7
4.1 Source consistency and scope 7
5 MATLAB Implementation 8
5.1 Initialization and sampling 8
5.2 Noise generation 8
5.3 Filter construction and plots 8
5.4 Reproduction environment 8
6 Simulation Results and Discussion 9
6.1 Original noisy and filtered signals 9
6.2 Overlay comparison 10
6.3 Delay startup and alignment 11
6.4 Frequency response and analytical predictions 12
6.5 Metric definitions and valid samples 13
6.6 Measured performance and interpretation 14
6.7 Comparison with analytical expectations 15
7 Limitations and Possible Improvements 16
8 Conclusion 16
References 17
Appendix Original MATLAB Source Code 18

1 Introduction

A measured or transmitted signal often contains random fluctuations that obscure its underlying waveform. Digital filtering operates on sampled data to reduce selected components of these fluctuations. A moving average filter is a useful introductory example because its coefficients, time-domain operation, and noise-reduction mechanism can be derived directly from an arithmetic mean.

The project contains README.md [7], which describes the demonstration, and Moving_Average_Noise_Filter.m [6], which implements it. The script creates a clean sinusoid, adds synthetic noise, and compares the noisy input with the filtered output. Since the clean signal and injected noise are known, residual noise can be measured separately from filter-induced signal distortion.

The executable script uses a 31-sample window, whereas README.md lists 15 samples in its parameter table and example. This report follows the implemented value N = 31. The difference is documented so that the results can be reproduced using the correct window length.

2 Project Objectives

The objectives are to generate a sampled 5 Hz sine wave, add Gaussian noise, apply a normalized moving average FIR filter, and compare the original, noisy, and filtered signals. The evaluation derives the expected delay and gain, measures noise reduction and reconstruction error, and documents the analysis procedure.

The evaluation asks whether smoothing improves the visibility and recovery of the sinusoid, how much residual noise remains, and how delay, attenuation, and startup affect that conclusion. It also checks that the numerical implementation uses the fixed averaging denominator and causal boundary conditions of MATLAB filter().

Although the README proposes investigating window size, the supplied script runs only one fixed window. No window sweep was performed for this report. The effect of window length is discussed analytically, and an experimental sweep is identified as possible future work.

3 Theoretical Background

3.1 Sampling and the original signal

Let t be time in seconds, Fs the sampling frequency in samples per second, f the sine-wave frequency in hertz, and n an integer sample index starting at zero. With sampling period Ts = 1/Fs, the project defines the time samples and original signal as follows.

$$T_s = \frac{1}{F_s}, t_n = \frac{n}{F_s}, x[n] = \sin(\frac{2\pi f n}{F_s})$$

For Fs = 1000 Hz, Ts is 0.001 s. The MATLAB vector t = 0:1/Fs:2 includes both endpoints, so n ranges from 0 through 2000 and the record contains 2001 samples. The sine wave has unit peak amplitude and f = 5 Hz. Its frequency is well below the Nyquist frequency of 500 Hz; the synthetic sinusoid is sampled without aliasing. Amplitude is dimensionless because the source assigns no physical voltage or other calibration.

3.2 Additive Gaussian noise

$$u[n] = x[n] + w[n], w[n] \sim \mathcal{N}(0, \sigma^2), \sigma = 0.6$$

Here u[n] is the noisy input x_noisy[n] in the source, w[n] is the additive noise sample, and σ is the population standard deviation. MATLAB randn generates standard normal samples, so multiplication by 0.6 gives variance $\sigma^2 = 0.36$ [2]. Gaussian samples are not bounded by ±0.6; the value is not a maximum noise amplitude. Zero mean is a distribution property, and a finite realization can have a nonzero sample mean. The variance model treats the samples as independent white noise.

3.3 Causal moving average filtering

$$y[n] = \frac{1}{N}\sum_{k=0}^{N-1} u[n-k]$$

The output y[n] averages N current and past input samples; k is the lag index. This makes the implementation causal. It is an FIR filter because its response to a single impulse lasts for a finite number of samples and the output does not depend recursively on past outputs [1, 4].

3.4 Impulse and frequency responses

The impulse response h[k] equals 1/N for k = 0 through N − 1 and zero otherwise. Its coefficients sum to one, giving unity gain for a constant input after startup. For angular frequency ω in radians per sample, the frequency response is the discrete-time Fourier transform of h[k].

$$H(e^{j\omega}) = \frac{1}{N}\sum_{k=0}^{N-1} e^{-j\omega k} = e^{-j\omega(N-1)/2}\frac{\sin(N\omega/2)}{N\sin(\omega/2)}$$

The symbol j denotes the imaginary unit, and a physical frequency F in hertz corresponds to ω = 2πF/Fs. At zero frequency the expression is interpreted by its limit, H(1) = 1. Magnitude is the absolute value of the sine ratio. The moving average has a low-pass main lobe with repeated zeros and sidelobes, so it does not suppress every high-frequency component equally [4].

3.5 Smoothing and delay

For independent zero-mean noise samples, averaging reduces variance because uncorrelated sample contributions add in power. The steady output-noise variance is σ^2 times the sum of squared coefficients, or σ^2/N. This is an ensemble prediction, not a guarantee for a single finite record. Adjacent output noise samples are correlated because their windows overlap.

The symmetric coefficients have linear phase, with group delay D = (N − 1)/2 away from response zeros [5]. For N = 31, D = 15 samples and D/Fs = 0.015 s = 15 ms. At 5 Hz, the theoretical amplitude gain is 0.960986, equivalent to -0.345660 dB, and the phase shift is −27°. The signal is smoothed and delayed, and its amplitude is slightly reduced.

Increasing N lowers white-noise variance but increases delay and moves the first spectral zero toward lower frequencies. Signal components closer to that zero are attenuated more strongly. A suitable window therefore depends on the useful signal bandwidth and allowable latency, not only on visual smoothness.

3.6 Startup under zero initial conditions

MATLAB filter() assumes zero initial delay states when none are supplied [1]. For n < 30, some of the 31 required past samples precede the record and are treated as zero. The divisor stays 31; the filter does not average only the available samples with a shorter denominator. The first complete window occurs at n = 30, or t = 0.030 s. The supplied script retains these startup samples and does not append the filter tail after the final input sample.

4 System Design and Simulation Parameters

The processing chain is signal generation, additive noise generation, normalized FIR filtering, and visual comparison. Table 1 records the settings verified directly against the original script. The evaluation additionally filters the clean signal and noise separately, without changing the baseline output, to identify their contributions.

Table 1. Verified implemented baseline

Parameter	Value or expression
Sampling frequency Fs	1000 Hz
Signal frequency f	5 Hz
Time vector	t = 0:1/Fs:2
Sample indices and count	n = 0…2000; 2001 samples
Original signal	x = sin(2*pi*f*t)
MATLAB random-state instruction	rng(1)
Added noise	0.6 * randn(size(x))
Noise standard deviation σ	0.6 amplitude units
Window length N	31 samples
Numerator coefficients b	ones(1,N)/N
Denominator coefficient a	1
Filtering operation	filter(b,1,x_noisy)

4.1 Source consistency and scope

README.md states N = 15 in both its simulation table and illustrative code, while Moving_Average_Noise_Filter.m assigns N = 31. The executable assignment is used throughout this report. The README phrase “Noise Amplitude” is interpreted correctly as the standard-deviation scale applied to randn. Neither source file was edited.

The original script creates two figures: a three-panel signal comparison and an overlay. The four figures and measured results in this report were generated using the Python reproduction. No hardware measurements, window-length sweep, or direct comparison with a native MATLAB run were performed.

5 MATLAB Implementation

5.1 Initialization and sampling

The script resets the display, workspace, and figures using clc, clear, and close all. It creates an endpoint-inclusive sampling grid with 2001 samples and the deterministic sine wave.

```
Fs = 1000;
t = 0:1/Fs:2;
f = 5;
x = sin(2*pi*f*t);
```

5.2 Noise generation

```
rng(1);
noise = 0.6 * randn(size(x));
x_noisy = x + noise;
```

The noise vector has the same size as x. MATLAB rng(1) seeds the current generator algorithm without explicitly selecting it [3]. Repeatability depends on the environment and generator; equal seeds across MATLAB, Octave, and Python do not establish identical sequences.

5.3 Filter construction and plots

```
N = 31;
b = ones(1, N) / N;
x_filtered = filter(b, 1, x_noisy);
```

The 31 coefficients each equal 1/31. With denominator 1, filter() implements the causal FIR sum [1]. The script produces three subplots and an overlay. The reproduction preserves both views and adds startup/alignment and frequency-response figures for explanation. Original source code remains unchanged.

5.4 Reproduction environment

Python 3.12.14 and NumPy 2.3.5 were used to verify the reported metrics. The figures show the corresponding Python simulation, plotted using Matplotlib. Noise uses numpy.random.RandomState(1).standard_normal with MT19937. These samples are not claimed to match the MATLAB random sequence.

The time grid is np.arange(2001)/1000. Causal filtering uses np.convolve(x_noisy, np.ones(31)/31, mode="full")[:2001]. Keeping the first 2001 output samples preserves the fixed divisor and zero initial conditions of MATLAB filter(). Sample count, startup behavior, linearity, and error decomposition were checked.

6 Simulation Results and Discussion

6.1 Original noisy and filtered signals

Figure 1 shows the full endpoint-inclusive record. The original trace is a smooth sinusoid. Adding Gaussian noise produces rapid fluctuations around it, and some excursions are much larger than the noise standard deviation. The causal filtered trace has fewer short-term variations, while the oscillatory structure remains visible.

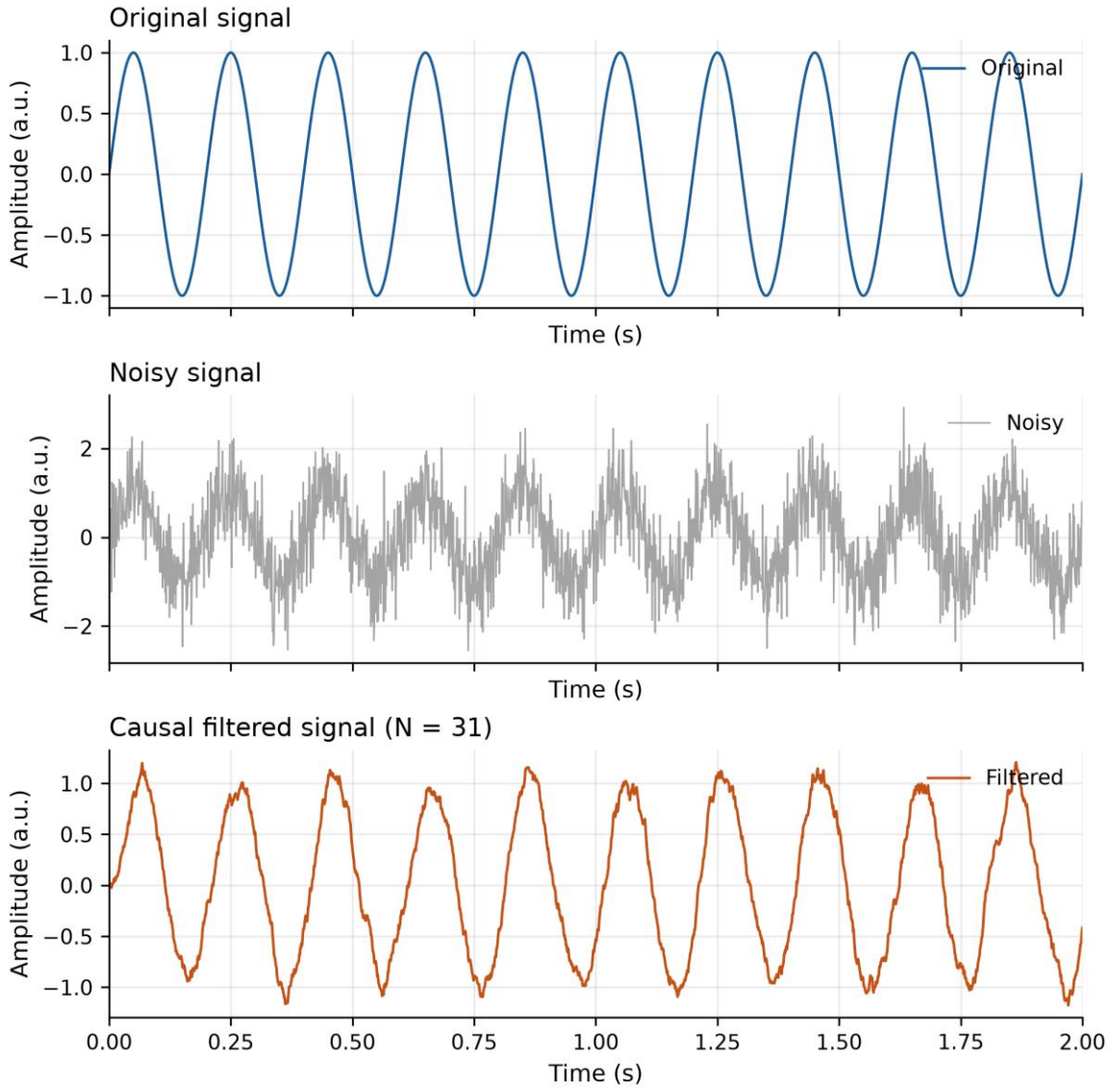


Figure 1. Original, noisy, and causal filtered signals for the Python reproduction. The filter uses N = 31 and zero initial conditions.

The filtered trace retains residual noise and its peaks occur later than the clean waveform. Blue denotes the original, gray the noisy input, orange the causal output, and green the aligned comparison in Figure 3.

6.2 Overlay comparison

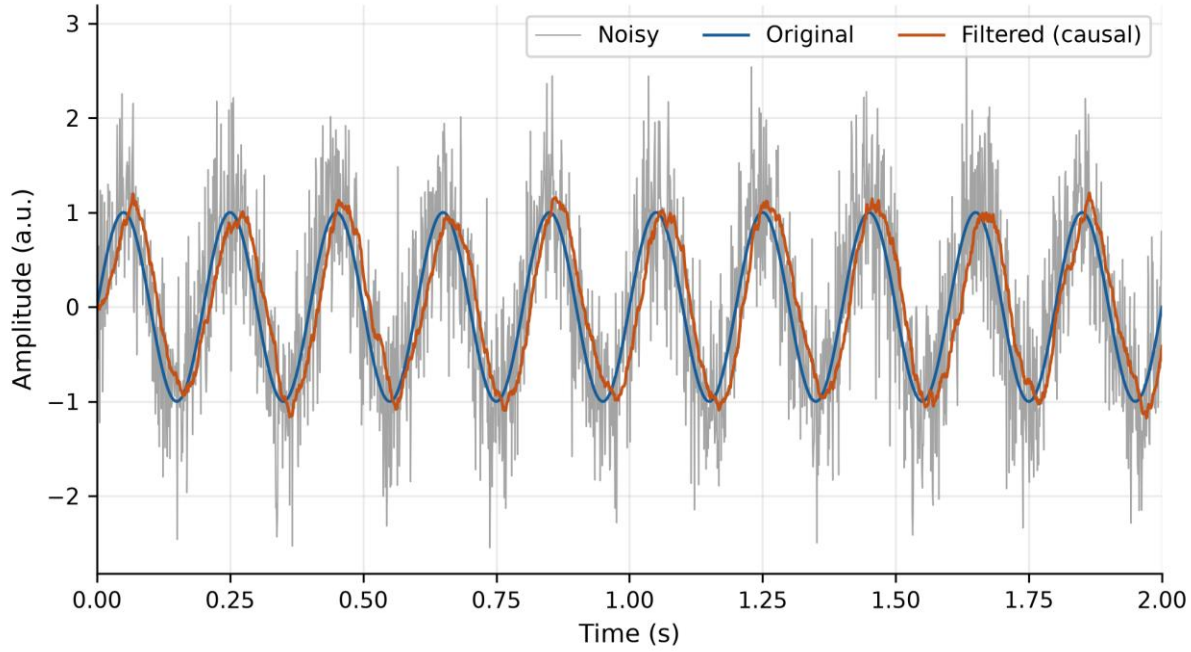


Figure 2. Overlay of the original signal, noisy input, and causal moving average output across the full record.

The overlay in Figure 2 makes the reduction of rapid fluctuations easier to see. The noisy input frequently obscures the clean waveform, whereas the filtered output follows its general shape. The remaining deviations include both a noise contribution and systematic filter effects. In particular, comparing the causal output directly against the original at the same timestamp includes phase delay as reconstruction error.

Noise smoothing is expected from averaging independent input fluctuations. It does not imply that every filtered sample is closer to the original than the corresponding noisy sample. Nor does a smoother curve establish preservation of all signal frequencies. The present result is specific to the low-frequency sine wave and this noise model.

The separated measured output noise power is 0.010309, while the theoretical steady white-noise power is 0.011613. Their difference is consistent with measuring one finite realization rather than an ensemble average. Overlapping windows also make the filtered noise samples dependent, so the nominal count of output samples is not a count of independent averages.

6.3 Delay startup and alignment

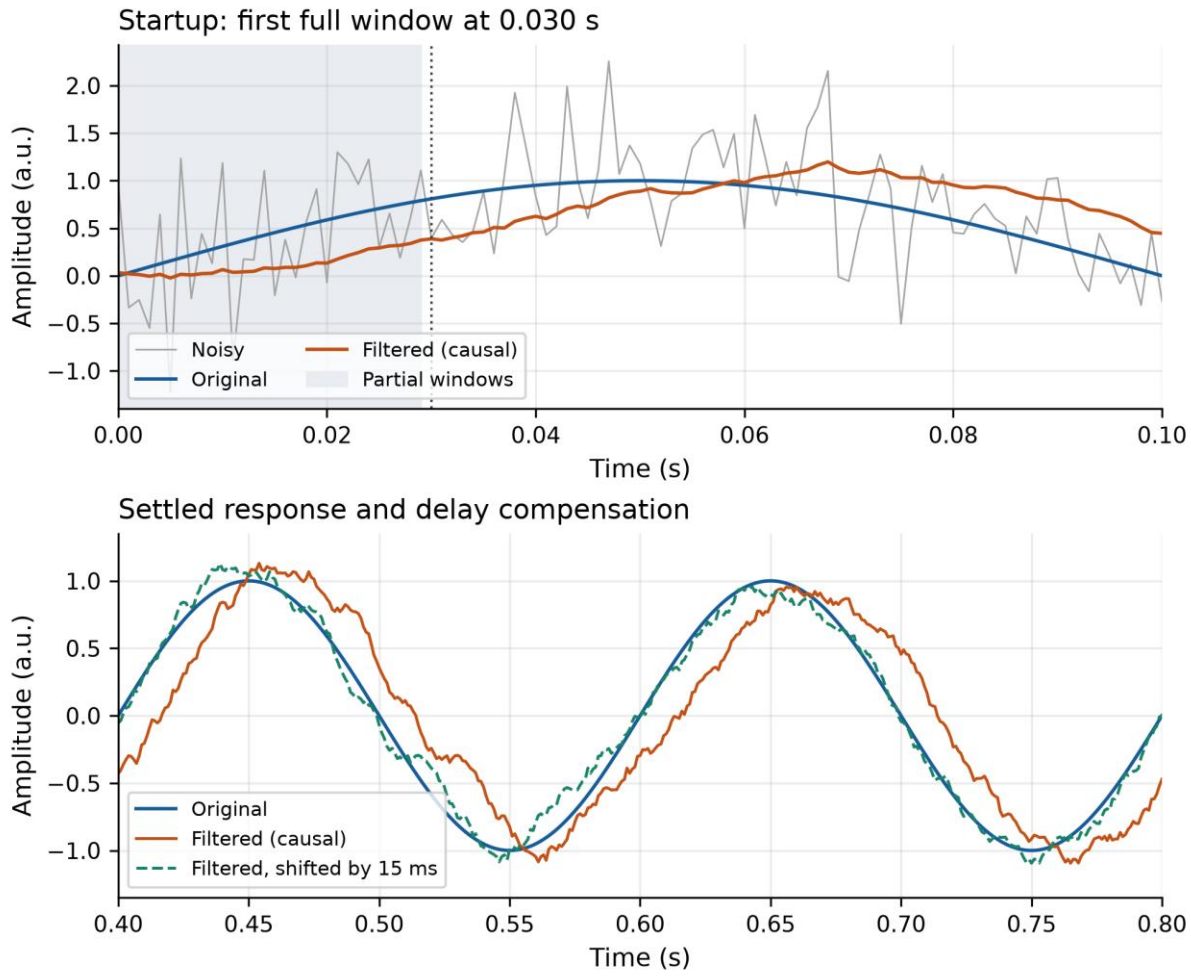


Figure 3. Startup with partial windows and a settled comparison showing the causal output and its 15 ms advance for evaluation.

The upper panel of Figure 3 isolates startup. Samples before n = 30 use partial data windows together with implicit zeros. This is a zero-state transient, not an adaptive change in window length. The lower panel shows a settled segment: shifting the output left by 15 samples removes the known time offset, while attenuation and residual noise remain.

Delay compensation is used only to assess recovery when timing is aligned. It requires a future output sample relative to each target timestamp and does not remove latency from a real-time implementation. It is not applied to the baseline MATLAB output or to Figures 1 and 2. All error metrics exclude partial-window outputs and unavailable boundary samples as defined in Section 6.5.

6.4 Frequency response and analytical predictions

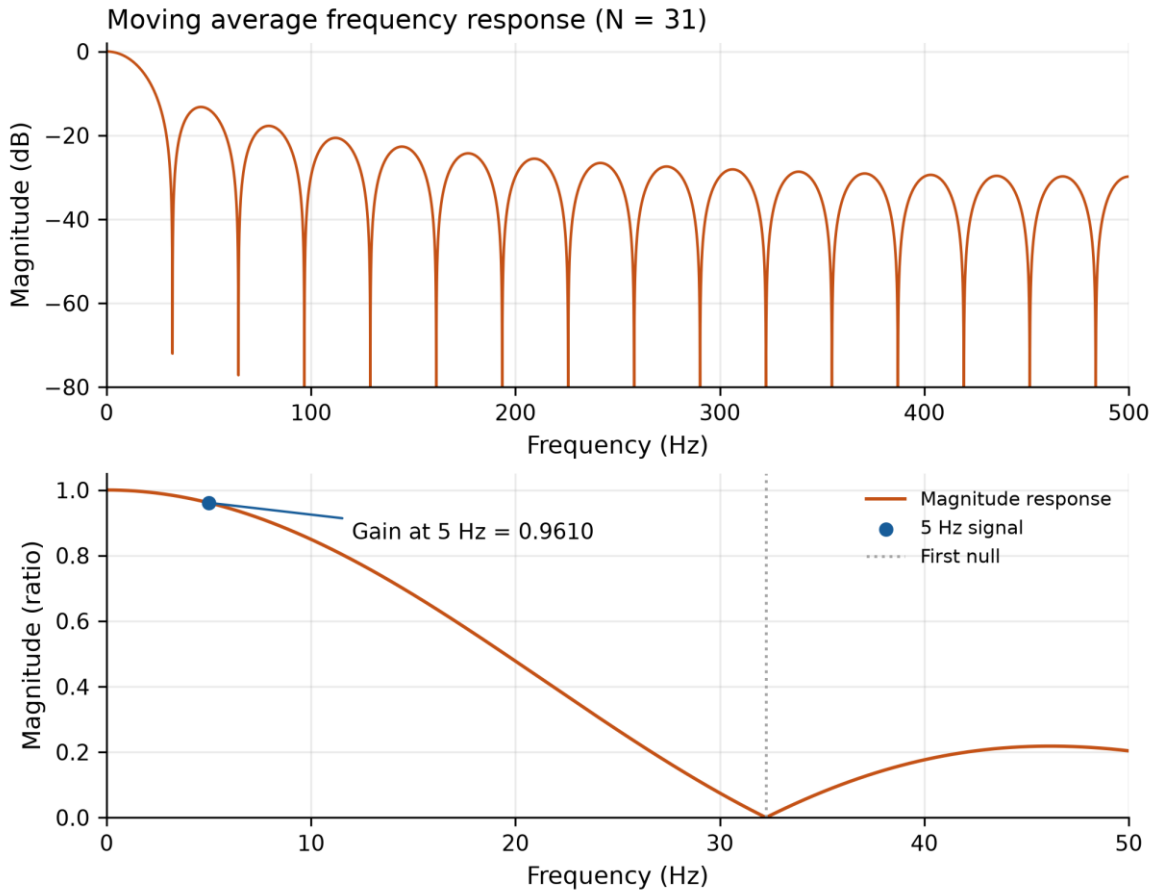


Figure 4. Analytical magnitude response of the implemented 31-tap coefficient vector, with a detailed view of the low-frequency main lobe.

Figure 4 is calculated from the filter response, not from an FFT estimate of the noisy realization. The first positive zero occurs at Fs/N = 32.258065 Hz. At 5 Hz the gain is 0.960986, corresponding to a 3.9014% amplitude reduction. The sidelobes show why a moving average offers limited stopband selectivity [4].

A least-squares sine/cosine fit to the separately filtered clean signal on the comparison interval gives gain 0.960986 and phase -27.0000°. This measured deterministic check agrees with the frequency-response prediction; it is not a fit to the noisy output.

6.5 Metric definitions and valid samples

All measured power and error values use the same target set I = {30, 31, …, 1985}, containing M = 1956 samples with target times from 0.030 to 1.985 s. Uncompensated comparisons use output indices 30 through 1985. The aligned comparison uses y[n + D], D = 15, with output indices 45 through 2000 and output times from 0.045 to 2.000 s. Every included output has a full averaging window; no extrapolation or circular shift is used.

Define s[n] as the clean signal filtered by h and v[n] as the injected noise filtered by h. Linearity gives y[n] = s[n] + v[n]. For any sequence z, P(z) denotes its mean square on the stated index set. It is not a sample variance with the empirical mean removed. All powers and MSE values have squared dimensionless-amplitude units.

$$P(z) = \frac{1}{M}\sum_{n\in I} z[n]^2\,, \mathrm{SNR}_{\mathrm{in}} = 10\log_{10}\frac{P(x)}{P(w)}, \mathrm{SNR}_{\mathrm{out}} = 10\log_{10}\frac{P(s)}{P(v)}$$

Input noise power is P(w), and output noise power is P(v). Output SNR uses the filtered clean-signal power P(s) in its numerator, so signal attenuation is included. It is not a reconstruction SNR that treats all differences from the original as noise.

$$\mathrm{MSE}_{\mathrm{before}} = P(u - x), \mathrm{MSE}_{\mathrm{raw}} = P(y - x)$$

$$\mathrm{MSE}_{\mathrm{aligned}} = \frac{1}{M}\sum_{n\in I}(y[n + D] - x[n])^2$$

Here, "before" compares the noisy input with the clean target, "raw" compares the causal output at its original timestamps, and "aligned" compensates for the known delay. Table 2 presents the measured results, and Table 3 presents the analytical predictions. All displayed values are rounded.

6.6 Measured performance and interpretation

Table 2. Measured Python baseline on the common target interval

Measure	Value	Unit
Input signal power P(x)	0.507123	amplitude²
Input noise power P(w)	0.365296	amplitude²
Input SNR	1.424679	dB
Filtered signal power P(s)	0.468325	amplitude²
Output noise power P(v)	0.010309	amplitude²
Output SNR	16.573486	dB
Noise power reduction	15.494466	dB
Reconstruction MSE before	0.365296	amplitude²
Reconstruction MSE raw	0.116688	amplitude²
Reconstruction MSE aligned	0.009557	amplitude²

The measured noise power reduction is 15.4945 dB, calculated as 10 log10(P(w)/P(v)). This confirms suppression of the injected random component on the specified interval. The remaining output noise is not zero, and a portion of the useful sine-wave amplitude is also lost.

The raw deterministic signal-distortion MSE, P(s − x), is 0.103838; after delay alignment it is 0.000772. The large reduction explains why timing accounts for much of the raw reconstruction error. This does not justify reporting the aligned metric as though the physical delay had been eliminated.

For this finite realization, raw reconstruction MSE equals distortion power 0.103838 plus noise power 0.010309 plus cross term 0.002541. The cross term is twice the sample mean of (s[n] − x[n])v[n]. It need not vanish exactly in a finite record, even though its ensemble expectation is zero for the assumed zero-mean noise.

In the aligned case, the corresponding values are 0.000772, 0.010333, and -0.001547. Their sum gives the aligned MSE in Table 2. The aligned noise samples occupy a different output index set, which is why their measured power differs slightly from the raw P(v).

6.7 Comparison with analytical expectations

Table 3. Analytical predictions for N = 31

Quantity	Prediction	Unit
Input noise variance	0.360000	amplitude²
Steady output noise power	0.011613	amplitude²
Steady output noise standard deviation	0.107763	amplitude
Noise power reduction	14.913617	dB
5 Hz amplitude gain	0.960986	ratio
5 Hz magnitude gain	-0.345660	dB
5 Hz phase shift	-27.000000	degrees
Group delay	15.000000	samples
Group delay at 1000 Hz	15.000000	ms
First positive spectral zero	32.258065	Hz

The theory predicts ensemble noise statistics and deterministic steady-state tone behavior. It does not predict the exact powers of this particular random vector. The comparison interval is also a subset of the full sine-wave record, so its measured signal power should be used in the SNR calculations instead of automatically substituting the infinite-duration sine-wave power.

The measured input noise sample mean is 0.021768. This does not contradict a zero-mean noise model. It is one finite realization, and the power metric intentionally retains any finite-record mean component. No multiple-seed confidence interval was calculated.

Agreement between the clean-tone fit and the analytical gain and phase checks the deterministic filter implementation. The output-noise measurement checks smoothing for the selected realization. Both are needed: visual smoothness alone does not quantify fidelity, and noise-power reduction alone does not quantify waveform delay or attenuation.

7 Limitations and Possible Improvements

The executed study uses one synthetic sine wave, one Gaussian noise realization, and one fixed window. It does not characterize performance for speech, sharp transitions, multiple tones, colored noise, or impulsive interference. A moving average has limited frequency selectivity, and suppressing rapid changes can remove useful features together with noise [4].

The causal implementation introduces latency and zero-state startup. In a stream, a valid initialization strategy would depend on what is known before the first sample. The present analysis excludes startup from quantitative comparisons but retains it in the plotted baseline. Offline alignment is an evaluation aid and cannot provide a zero-latency causal output.

The Python reproduction validates the deterministic model and filter boundary conditions, but it does not establish equality to a MATLAB Gaussian sequence. A future native MATLAB run should export the injected noise and results, record the MATLAB release and generator type, and recalculate the same metrics. That would enable an explicit numerical comparison between environments.

Possible improvements are a controlled sweep of window lengths, repeated trials over independent seeds, and comparison with a designed low-pass FIR filter when a specified transition band or stopband attenuation is required. Any gain correction should be assessed for its effect on residual noise. Alternative startup handling and offline zero-phase filtering would change the implemented behavior and should be reported as separate experiments. None of these extensions was performed here.

8 Conclusion

The supplied MATLAB project implements a normalized 31-sample causal FIR moving average. In the disclosed Python reproduction, output noise power decreases to 0.010309 on the defined comparison interval, and the output SNR is 16.5735 dB. The filtered waveform retains the overall sinusoidal shape while reducing rapid random fluctuations.

This improvement includes a measurable fidelity trade-off. The filter delays the waveform by 15 ms and has a theoretical 5 Hz gain of 0.960986. Reconstruction MSE is 0.116688 at the original timestamps and 0.009557 after explicitly documented alignment. Separating residual noise from delay and attenuation makes the result interpretable and provides a sound basis for future window selection.

References

[1] The MathWorks, Inc. “filter — 1-D digital filter.” MATLAB Documentation. Accessed 1 November 2026. https://www.mathworks.com/help/matlab/ref/filter.html

[2] The MathWorks, Inc. “randn — Normally distributed random numbers.” MATLAB Documentation. Accessed 1 November 2026. https://www.mathworks.com/help/matlab/ref/double.randn.html

[3] The MathWorks, Inc. “rng — Control random number generator.” MATLAB Documentation. Accessed 1 November 2026. https://www.mathworks.com/help/matlab/ref/rng.html

[4] S. W. Smith, The Scientist and Engineer’s Guide to Digital Signal Processing. San Diego, CA: California Technical Publishing, 1997, Chapter 15, “Moving Average Filters,” pp. 277–284. Accessed 1 November 2026. https://www.dspguide.com/CH15.PDF. Edition details: https://www.dspguide.com/editions.htm

[5] The MathWorks, Inc. “Compensate for the Delay Introduced by an FIR Filter.” MATLAB Documentation. Accessed 1 November 2026. https://www.mathworks.com/help/signal/ug/compensate-for-the-delay-introduced-by-an-fir-filter.html

[6] Project source. Moving_Average_Noise_Filter.m. MATLAB script supplied with the project; the complete code is reproduced in the Appendix.

[7] Project documentation. README.md. Project overview, stated simulation parameters, and illustrative code supplied with the project.

Appendix Original MATLAB Source Code

Complete original script from Moving_Average_Noise_Filter.m [6].

```
clc;
clear;
close all;

% 1. Generate the original signal
Fs = 1000;                  % Sampling frequency (Hz)
t = 0:1/Fs:2;               % Time from 0 to 2 seconds
f = 5;                      % Signal frequency (Hz)

x = sin(2*pi*f*t);          % Original sine signal

% 2. Add random noise
rng(1);                     % Make the result repeatable
noise = 0.6 * randn(size(x));
x_noisy = x + noise;

% 3. Apply Moving Average Filter
N = 31;                     % Moving average window size
b = ones(1, N) / N;         % Filter coefficients

x_filtered = filter(b, 1, x_noisy);

% 4. Plot the results
figure('Name', 'Moving Average Noise Filter');

subplot(3,1,1);
plot(t, x, 'b', 'LineWidth', 1.2);
title('Original Signal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(3,1,2);
plot(t, x_noisy, 'r');
title('Noisy Signal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(3,1,3);
plot(t, x_filtered, 'k', 'LineWidth', 1.2);
title(['Filtered Signal (N = ', num2str(N), ')']);
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

sgtitle('Moving Average Noise Filter');

% 5. Compare noisy and filtered signals
figure('Name', 'Filter Comparison');
plot(t, x_noisy, 'Color', [0.7 0.7 0.7]);
hold on;
plot(t, x_filtered, 'b', 'LineWidth', 1.2);
plot(t, x, 'r--', 'LineWidth', 1.2);
title('Comparison of Original, Noisy and Filtered Signals');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Noisy Signal', 'Filtered Signal', 'Original Signal');
grid on;
```